

Далее мы собираемся настроить приложение для управления проектами [Redmine](#), базу данных PostgreSQL и [Adminer](#), графический интерфейс для администрирования баз данных.

Все вышеперечисленное имеет официальные Docker образы, доступные, как мы можем видеть из [Redmine](#), [Postgres](#) и [Adminer](#) соответственно. Официальность контейнеров не так важна, просто мы можем ожидать, что у них будет некоторая поддержка. Мы также могли бы, например, настроить Wordpress или MediaWiki внутри контейнеров тем же способом, если вас интересует запуск существующих приложений внутри Docker. Вы даже могли бы настроить инструмент мониторинга приложений, такой как [Sentry](#).

В https://hub.docker.com/_/redmine есть список различных помеченных версий:

Supported tags and respective Dockerfile links

- [5.1.2](#), [5.1](#), [5](#), [latest](#), [5.1.2-bookworm](#), [5.1-bookworm](#), [5-bookworm](#), [bookworm](#)
- [5.1.2-alpine3.18](#), [5.1-alpine3.18](#), [5-alpine3.18](#), [alpine3.18](#), [5.1.2-alpine](#), [5.1-alpine](#), [5-alpine](#), [alpine](#)
- [5.0.8](#), [5.0](#), [5.0.8-bookworm](#), [5.0-bookworm](#)
- [5.0.8-alpine3.18](#), [5.0-alpine3.18](#), [5.0.8-alpine](#), [5.0-alpine](#)

Скорее всего, мы можем использовать любой из доступных образов.

Из раздела *Environment Variables* мы можем видеть, что все версии могут использовать переменную окружения `REDMINE_DB_POSTGRES` для настройки базы данных Postgres. Итак, прежде чем двигаться дальше, давайте настроим Postgres для нас.

В https://hub.docker.com/_/postgres есть пример compose файла в разделе "via docker-compose or docker stack deploy". Давайте упростим его следующим образом:

```
version: "3.8"

services:
  db:
    image: postgres:13.2-alpine
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: example
    container_name: db_redmine
```

Примечание:

- `restart: always` было изменено на `unless-stopped`, что будет держать контейнер работающим, если мы явно не остановим его. С `always` остановленный контейнер запускается после перезагрузки, например, см. [здесь](#) для получения дополнительной информации.

В разделе [Where to store data](#) мы можем видеть, что `/var/lib/postgresql/data` должна быть смонтирована отдельно для сохранения данных.

Есть два варианта выполнения монтирования. Мы могли бы использовать bind mount, как ранее, и смонтировать легко находимую директорию для хранения данных. Давайте теперь используем другой вариант, [Docker managed volume](#).

Давайте запустим Docker Compose файл, не устанавливая ничего нового:

```
$ docker compose up

✓ Network redmine_default Created
0.0s
✓ Container db_redmine Created
0.2s
Attaching to db_redmine
db_redmine | The files belonging to this database system will be owned by user
"postgres".
db_redmine | This user must also own the server process.
...
db_redmine | 2024-03-11 14:05:52.340 UTC [1] LOG: starting PostgreSQL 13.2 on
aarch64-unknown-linux-musl, compiled by gcc (Alpine 10.2.1_pre1) 10.2.1 20201203,
64-bit
db_redmine | 2024-03-11 14:05:52.340 UTC [1] LOG: listening on IPv4 address
"0.0.0.0", port 5432
db_redmine | 2024-03-11 14:05:52.340 UTC [1] LOG: listening on IPv6 address
"::", port 5432
db_redmine | 2024-03-11 14:05:52.342 UTC [1] LOG: listening on Unix socket
"/var/run/postgresql/.s.PGSQL.5432"
db_redmine | 2024-03-11 14:05:52.345 UTC [46] LOG: database system was shut down
at 2024-03-11 14:05:52 UTC
db_redmine | 2024-03-11 14:05:52.347 UTC [1] LOG: database system is ready to
accept connections
```

Образ инициализирует файлы данных при первом запуске. Давайте завершим контейнер с помощью ^C. Compose использует текущую директорию как префикс для имен контейнеров и томов, чтобы разные проекты не конфликтовали (Префикс можно переопределить с помощью переменной окружения `COMPOSE_PROJECT_NAME`, если нужно).

Давайте **проверим**, был ли создан том с помощью `docker container inspect db_redmine | grep -A 5 Mounts`

```
"Mounts": [
  {
    "Type": "volume",
    "Name": "2d86a2480b60743147ce88e8e70b612d10b4c4151779b462baf4e81b84061ef5",
    "Source":
"/var/lib/docker/volumes/2d86a2480b60743147ce88e8e70b612d10b4c4151779b462baf4e81b84061ef5/_data",
    "Destination": "/var/lib/postgresql/data",
```

И действительно, он есть! Так что, несмотря на то, что мы **не** настраивали его явно, анонимный том был автоматически создан для нас.

Теперь, если мы проверим `docker volume ls`, мы можем видеть, что том с именем "2d86a2480b60743147ce88e8e70b612d10b4c4151779b462baf4e81b84061ef5" существует.

```
$ docker volume ls
DRIVER          VOLUME NAME
local          2d86a2480b60743147ce88e8e70b612d10b4c4151779b462baf4e81b84061ef5
```

На вашей машине может быть больше томов. Если вы хотите избавиться от них, вы можете использовать `docker volume prune`. Давайте теперь остановим все "приложение" с помощью `docker compose down`.

Вместо случайно именованного тома нам лучше определить его явно. Давайте изменим определение следующим образом:

```
version: "3.8"

services:
  db:
    image: postgres:13.2-alpine
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: example
    container_name: db_redmine
    volumes:
      - database:/var/lib/postgresql/data

volumes:
  database:
```

Теперь, после повторного запуска `docker compose up`, давайте посмотрим, как это выглядит:

```
$ docker volume ls
DRIVER          VOLUME NAME
local          redmine_database

$ docker container inspect db_redmine | grep -A 5 Mounts
"Mounts": [
  {
    "Type": "volume",
    "Name": "redmine_database",
    "Source": "/var/lib/docker/volumes/ongoing_redminedata/_data",
    "Destination": "/var/lib/postgresql/data",
```

Хорошо, выглядит немного более читаемо для человека! Теперь, когда Postgres работает, пришло время добавить [Redmine](#).

Контейнер, похоже, требует только двух переменных окружения.

```
redmine:
  image: redmine:5.1-alpine
```

```
environment:
  - REDMINE_DB_POSTGRES=db
  - REDMINE_DB_PASSWORD=example
ports:
  - 9999:3000
depends_on:
  - db
```

Обратите внимание на объявление [depends_on](#). Это гарантирует, что сервис `db` запустится первым. `depends_on` не гарантирует, что база данных запущена, только то, что она запущена первой. Сервер Postgres доступен с DNS именем "db" из сервиса Redmine, как обсуждалось в разделе [Docker networking](#).

Теперь, когда вы запускаете `docker compose up`, вы увидите множество миграций базы данных, выполняющихся сначала.

```
redmine_1 | I, [2024-03-03T10:59:20.956936 #25] INFO -- : Migrating to Setup (1)
redmine_1 | == 1 Setup: migrating
=====
...
redmine_1 | [2024-03-03 11:01:10] INFO  ruby 3.2.3 (2024-01-30) [x86_64-linux]
redmine_1 | [2024-03-03 11:01:10] INFO  WEBrick::HTTPServer#start: pid=1
port=3000
```

Как упоминается в [документации](#), образ создает файлы в `/usr/src/redmine/files`, и их лучше сохранить. Dockerfile имеет эту [строку](#), где он объявляет, что должен быть создан том. Снова Docker создаст том, но он будет обрабатываться как анонимный том, не управляемый Docker Compose, так что лучше создать его явно.

С учетом этого, наша конфигурация меняется на это:

```
version: "3.8"

services:
  db:
    image: postgres:13.2-alpine
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: example
    container_name: db_redmine
    volumes:
      - database:/var/lib/postgresql/data
  redmine:
    image: redmine:4.1-alpine
    environment:
      - REDMINE_DB_POSTGRES=db
      - REDMINE_DB_PASSWORD=example
    ports:
      - 9999:3000
    volumes:
      - files:/usr/src/redmine/files
```

```
  depends_on:
    - db

volumes:
  database:
  files:
```

Теперь мы можем использовать приложение с помощью нашего браузера через <http://localhost:9999>. После некоторых изменений внутри приложения мы можем проверить изменения, произошедшие в образе, и убедиться, что никакие дополнительные значимые файлы не были записаны в контейнер:

```
$ docker container diff $(docker compose ps -q redmine)
C /usr/src/redmine/config/environment.rb
...
C /usr/src/redmine/tmp/pdf
```

Вероятно, нет.

Мы могли бы использовать команду `psql` внутри контейнера Postgres для взаимодействия с базой данных, запустив

```
docker container exec -it db_redmine psql -U postgres
```

Тот же метод можно использовать для создания резервных копий с помощью `pg_dump`

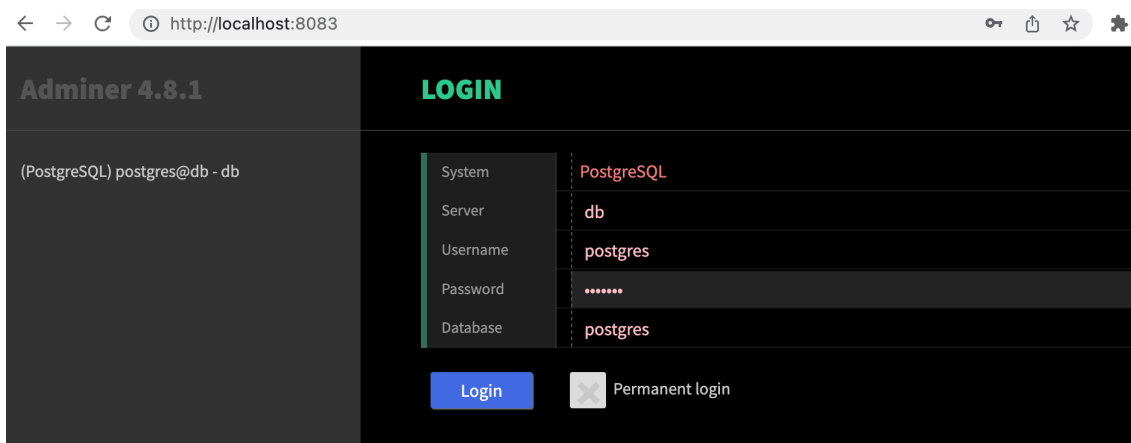
```
docker container exec db_redmine pg_dump -U postgres > redmine.dump
```

Вместо использования архаичного интерфейса командной строки для доступа к Postgres, давайте теперь настроим базу данных [Adminer](#) для приложения.

После просмотра [документации](#), настройка проста:

```
adminer:
  image: adminer:4
  restart: always
  environment:
    - ADMINER_DESIGN=galkaev
  ports:
    - 8083:8080
```

Теперь, когда мы запускаем приложение, мы можем получить доступ к adminer с <http://localhost:8083>:



Настройка adminer проста, так как он сможет получить доступ к базе данных через Docker сеть. Вы можете задаться вопросом, как adminer находит контейнер базы данных Postgres. Мы предоставляем эту информацию Redmine, используя переменную окружения:

```
redmine:
  environment:
    - REDMINE_DB_POSTGRES=db
```

Adminer фактически предполагает, что база данных имеет DNS имя *db*, так что с этим выбором имени нам не пришлось ничего указывать. Если база данных имеет какое-то другое имя, мы должны передать его adminer, используя переменную окружения:

```
adminer:
  environment:
    - ADMINER_DEFAULT_SERVER=database_server
```

Упражнения 2.6 - 2.10

:::info Упражнение 2.6

Давайте продолжим с примером приложения, с которым мы работали в [Упражнении 2.4](#).

Теперь вы должны добавить базу данных к примеру backend.

Используйте базу данных Postgres для сохранения сообщений. Пока нет необходимости настраивать том, так как официальный образ Postgres устанавливает том по умолчанию для нас. Используйте документацию образа Postgres в свою пользу при настройке: https://hub.docker.com/_/postgres/. Особенно ценна часть *Environment Variables*.

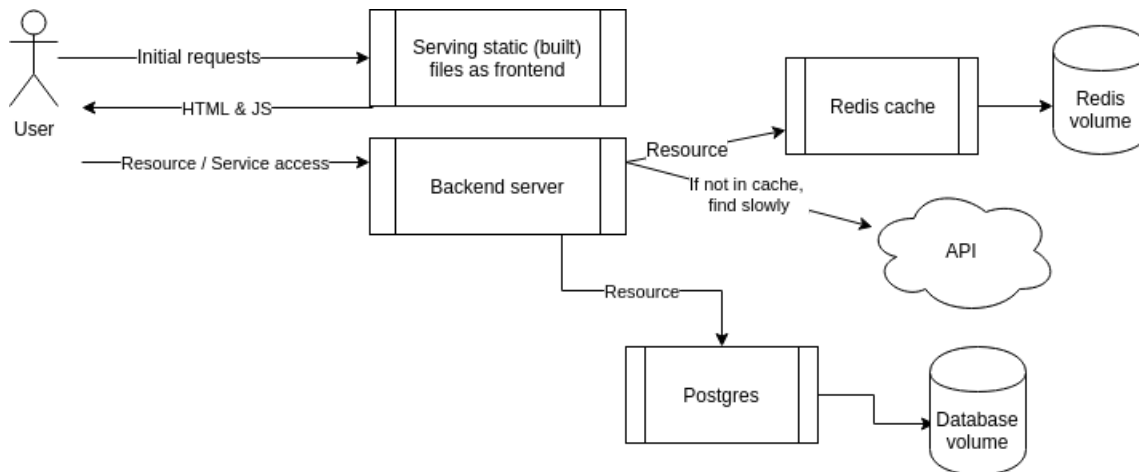
Backend [README](#) должен содержать всю информацию, необходимую для подключения.

Снова есть кнопка (и форма!) в frontend, которую вы можете использовать, чтобы убедиться, что ваша конфигурация выполнена правильно.

Отправьте docker-compose.yml

СОВЕТЫ:

- При настройке базы данных вам может понадобиться уничтожить автоматически созданные тома. Используйте команды `docker volume prune`, `docker volume ls` и `docker volume rm`, чтобы удалить неиспользуемые тома при тестировании. Убедитесь, что вы удалили контейнеры, которые от них зависят, заранее.
- `restart: unless-stopped` может помочь, если Postgres требуется некоторое время для готовности



...

...info Упражнение 2.7

Образ Postgres использует том по умолчанию. Определите ручную том для базы данных в удобном месте, таком как `./database`, так что вы должны использовать [bind mount](#). [Документация](#) образа может помочь вам с задачей.

После того, как вы настроили bind mount том:

- Сохраните несколько сообщений через frontend
- Запустите `docker compose down`
- Запустите `docker compose up` и убедитесь, что сообщения доступны после обновления браузера
- Запустите `docker compose down` и удалите папку тома вручную
- Запустите `docker compose up` и данные должны исчезнуть

СОВЕТ: Чтобы избавить вас от необходимости тестировать все эти шаги, просто загляните в папку перед попыткой шагов. Если она пуста после `docker compose up`, то что-то не так.

Отправьте `docker-compose.yml`

Преимущество bind mount в том, что, поскольку вы точно знаете, где находятся данные в вашей файловой системе, легко создавать резервные копии. Если используются управляемые Docker тома, расположение данных в файловой системе нельзя контролировать, и это делает резервные копии немного менее тривиальными...

...

...tip Советы для обеспечения работы подключения backend

В следующем упражнении попробуйте использовать ваш браузер для доступа к <http://localhost/api/ping> и посмотрите, отвечает ли он pong

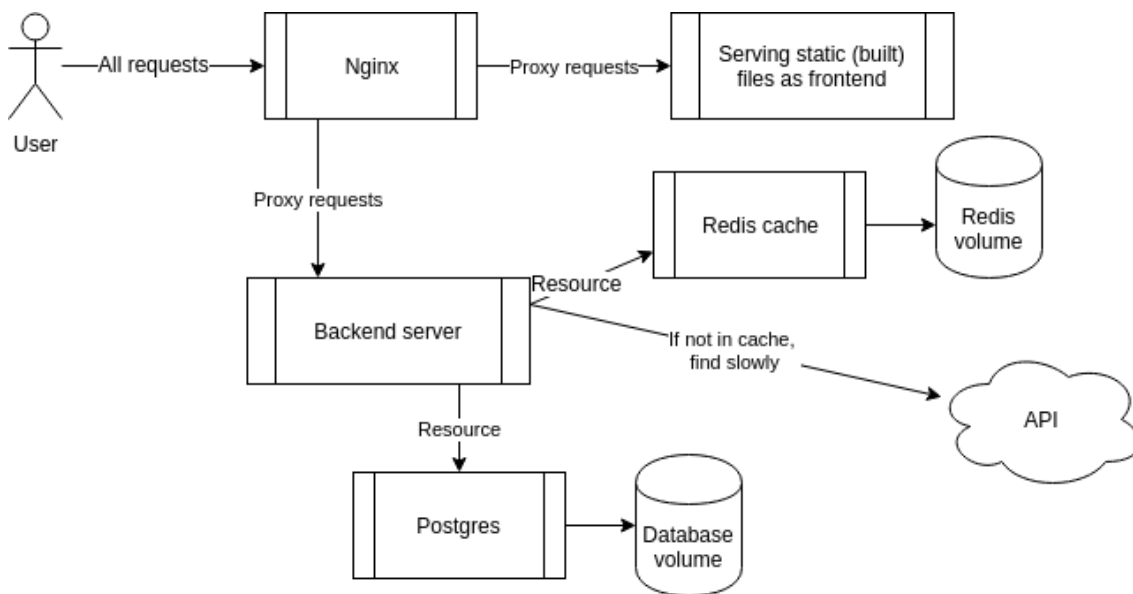
Это может быть проблема конфигурации Nginx. Убедитесь, что есть завершающий / на URL backend, как указано в контексте location /api/ в nginx.conf.

...

:::info Упражнение 2.8

В этом упражнении вы должны добавить [Nginx](#) для работы в качестве [обратного прокси](#) перед примером frontend и backend приложения.

Согласно Wikipedia [обратный прокси](#) — это тип прокси-сервера, который извлекает ресурсы от имени клиента с одного или нескольких серверов. Эти ресурсы затем возвращаются клиенту, выглядя так, как будто они исходят от самого обратного прокси-сервера.



Итак, в нашем случае обратный прокси будет единой точкой входа в наше приложение, и конечной целью будет размещение React frontend и Express backend за обратным прокси.

Идея в том, что браузер делает *все* запросы на <http://localhost>. Если запрос имеет префикс URL <http://localhost/api>, Nginx должен перенаправить запрос на backend контейнер. Все остальные запросы направляются на frontend контейнер.

Итак, в конце вы должны видеть, что frontend доступен просто переходом на <http://localhost>. Все кнопки, кроме той, что помечена *Упражнение 2.8*, могут перестать работать, не беспокойтесь о них, мы исправим это позже.

Следующий файл должен быть установлен в `/etc/nginx/nginx.conf` внутри Nginx контейнера. Вы можете использовать bind mount файла, где содержимое файла следующее:

```
events { worker_connections 1024; }

http {
    server {
```



```

listen 80;

location / {
    proxy_pass _frontend-connection-url_;
}

# configure here where requests to http://localhost/api/...
# are forwarded
location /api/ {
    proxy_set_header Host $host;
    proxy_pass _backend-connection-url_;
}
}
}

```

Nginx, backend и frontend должны быть подключены в одной сети. См. изображение выше для понимания того, как сервисы связаны. Вы можете найти [документацию Nginx](#) полезной, но помните, конфигурация, которая вам нужна, довольно проста, если вы делаете сложные вещи, вы, скорее всего, делаете что-то неправильно.

Если и когда ваше приложение "не работает", помните, что нужно посмотреть в лог, это может быть очень полезно для выявления ошибок:

```

2_7-proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-
2_7-proxy-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
2_7-proxy-1 | 2023/03/05 09:24:51 [emerg] 1#1: invalid URL prefix in
/etc/nginx/nginx.conf:8
2_7-proxy-1 exited with code 1

```

Отправьте docker-compose.yml

...

...:info Упражнение 2.9

Большинство кнопок могли перестать работать в примере приложения. Убедитесь, что каждая кнопка для упражнений работает.

Помните, что нужно заглянуть в консоли разработчика браузера снова, как мы делали в [части 1](#), помните также [это](#) и [это](#).

Кнопки упражнения Nginx и первая кнопка ведут себя по-разному, но вы хотите, чтобы они совпадали.

Если вам пришлось внести какие-либо изменения, объясните, что вы сделали и где.

Отправьте docker-compose.yml и оба Dockerfiles.

...

...:tip Публикация портов в хост-сеть

Есть важный урок о Docker сетях и портах, который нужно усвоить в следующем упражнении.

Когда мы делаем [сопоставление портов](#), в `docker run -p 8001:80 ...` или в файле Docker Compose, мы [публикуем](#) порт контейнера в хост-сеть, чтобы он был доступен на localhost.

Порт контейнера доступен внутри Docker сети для других контейнеров, находящихся в той же сети, даже если мы ничего не публикуем. Таким образом, публикация портов нужна только для открытия портов за пределами Docker сети. Если прямой доступ за пределы сети не нужен, то мы просто ничего не публикуем. :::

:::info Упражнение 2.10

Теперь у нас есть обратный прокси, работающий! Все коммуникации с нашим приложением должны проходить через обратный прокси, и прямой доступ (например, доступ к backend с GET на <http://localhost:8080/ping>) должен быть предотвращен.

Используйте сканер портов, например <https://hub.docker.com/r/networkstatic/nmap>, чтобы убедиться, что нет лишних открытых портов на хосте.

Может быть достаточно просто запустить

```
$ docker run -it --rm --network host networkstatic/nmap localhost
```

Если у вас Mac M1/M2, вам может понадобиться собрать образ самостоятельно.

Результат выглядит следующим образом (я использовал самособранный образ):

```
$ docker run -it --rm --network host nmap localhost
Starting Nmap 7.93 ( https://nmap.org ) at 2023-03-05 12:28 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 996 closed tcp ports (reset)
PORT      STATE      SERVICE
80/tcp    filtered  http
111/tcp   open      rpcbind
5000/tcp  filtered  complex-link
8080/tcp  filtered  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 1.28 seconds
```

Как мы видим, есть два подозрительных открытых порта: 5000 и 8080. Так что очевидно, что frontend и backend все еще напрямую доступны в хост-сети. Это нужно исправить!

Вы закончите, когда отчет сканирования портов будет выглядеть примерно так:

```
Starting Nmap 7.93 ( https://nmap.org ) at 2023-03-05 12:39 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (reset)
PORT      STATE      SERVICE
80/tcp    filtered  http
111/tcp   open      rpcbind
```

Nmap **done**: 1 IP address (1 host up) scanned **in** 1.28 seconds

...